

File Handling

File handling means working with files using Python. We can use Python to **create, read, write, append, and update** files.

The main function used for file handling is:

`open()`

Opening a File

Syntax

```
file = open("filename.txt", "mode")
```

Example:

```
file = open("sample.txt", "r")
```

Here:

- `sample.txt` → File name
 - `"r"` → Mode
-

File Modes

Mode	Meaning
"r"	Read
"w"	Write
"a"	Append
"x"	Create

"r" – Read

Opens a file for reading.

```
file = open("sample.txt", "r")
```

If the file doesn't exist, an error occurs.

"w" – Write

Opens a file for writing.

```
file = open("sample.txt", "w")
```

If the file doesn't exist, Python creates it.

Important: Existing content can be overwritten.

"a" – Append

Adds new content at the end of the file.

```
file = open("sample.txt", "a")
```

"x" – Create

Creates a new file.

```
file = open("sample.txt", "x")
```

If the file already exists, an error occurs.

These are the four basic file modes covered in your uploaded notes.

Writing to a File

Use `write()` to write data.

```
file = open("sample.txt", "w")
```

```
file.write("Hello Python")
file.write("\nWelcome to File Handling")
```

```
file.close()
```

This creates sample.txt and writes the text into it.

Reading a File

Use read() to read the complete file.

```
file = open("sample.txt", "r")
data = file.read()
print(data)
file.close()
```

readline()

readline() reads **one line** from a file.

```
file = open("sample.txt", "r")
print(file.readline())
file.close()
```

readlines()

readlines() reads all lines and returns them as a list.

```
file = open("sample.txt", "r")
lines = file.readlines()
print(lines)
file.close()
```

Your uploaded notes specifically include read(), readline(), and readlines() for reading files.

Appending to a File

Use "a" mode when you want to add content without removing existing content.

```
file = open("sample.txt", "a")
file.write("\nThis is a new line")
file.close()
```

with open()

A better way to work with files is using with.

with open("sample.txt", "r") as file:

```
    data = file.read()
print(data)
```

The file is automatically closed after the with block.

Writing Multiple Lines

with open("sample.txt", "w") as file:

```
file.write("Python\n")
file.write("Java\n")
file.write("C++\n")
```

writelines()

writelines() can write multiple strings to a file.

```
lines = [
    "Python\n",
    "Java\n",
    "C++\n"
]
with open("sample.txt", "w") as file:
    file.writelines(lines)
```

Updating a File

To update a particular line, one simple approach is:

```
with open("sample.txt", "r") as file:
    lines = file.readlines()
```

```
lines[1] = "JavaScript\n"
```

```
with open("sample.txt", "w") as file:
    file.writelines(lines)
```

Here:

1. File is opened in read mode.
2. Lines are stored in a list.
3. Required line is changed.
4. File is opened in write mode.
5. Updated lines are written back.

This update approach is also shown in your uploaded notes.

Closing a File

When using open() directly, use:

```
file.close()
```

Example:

```
file = open("sample.txt", "r")
data = file.read()
print(data)
file.close()
```

When using with open(), Python handles closing automatically.

Deleting a File

Python's `os` module can be used to delete a file.

```
import os
os.remove("sample.txt")
```

It's safer to check whether the file exists first:

```
import os
if os.path.exists("sample.txt"):
    os.remove("sample.txt")
    print("File deleted")
else:
    print("File does not exist")
```

This deletion approach is included in your uploaded file-handling notes.

Error Handling

Error handling is used to handle errors that occur while a Python program is running.

Instead of allowing the program to stop suddenly, we can use error-handling statements to handle the error properly.

The main keywords are:

```
try
except
else
finally
raise
```

Why Do We Need Error Handling?

Consider this program:

```
a = int(input("Enter a number: "))
b = int(input("Enter another number: "))
```

```
print(a / b)
```

If the user enters:

10

0

Python produces:

`ZeroDivisionError`

The program stops.

With error handling, we can handle the problem.

Errors and Their Types in Python

Errors in Python occur when the interpreter encounters something it cannot execute. Errors can be broadly classified into **syntax errors** and **exceptions**.

1. Syntax Errors (Compile-Time Errors)

A **SyntaxError** occurs when the Python interpreter finds an incorrect syntax (wrong grammar of Python).

Example:

```
if True
    print("Hello")
```

Error Output:

SyntaxError: expected ':'

✓ **Fix:** Add a colon : after if True.

2. Exceptions (Runtime Errors)

Exceptions occur **at runtime** when a valid syntax is executed but results in an error.

Types of Exceptions in Python

Here are some common types of exceptions:

(i) NameError

Occurs when trying to use a variable that is not defined.

```
print(x) # x is not defined
```

Error Output:

NameError: name 'x' is not defined

✓ **Fix:** Define x before using it.

(ii) TypeError

Occurs when an operation is performed on an incompatible type.

```
print(5 + "hello") # Integer + String
```

Error Output:

TypeError: unsupported operand type(s) for +: 'int' and 'str'

✓ **Fix:** Convert data types before operations:

```
print(str(5) + "hello") # Output: 5hello
```

(iii) ValueError

Occurs when a function receives an argument of the correct type but an inappropriate value.

```
num = int("abc") # Cannot convert "abc" to an integer
```

Error Output:

ValueError: invalid literal for int() with base 10: 'abc'

✓ **Fix:** Ensure valid input before conversion.

(iv) IndexError

Occurs when trying to access an index that is out of range.

```
lst = [1, 2, 3]
print(lst[5]) # Index out of
range
```

Error Output:

IndexError: list index out of range

✓ **Fix:** Check list length before accessing indexes.

(v) KeyError

Occurs when trying to access a dictionary key that does not exist.

```
my_dict = {"a": 1, "b": 2}
print(my_dict["c"]) # Key 'c' does not exist
```

Error Output:

KeyError: 'c'

✓ **Fix:** Use `.get()` to handle missing keys:
`print(my_dict.get("c", "Key not found"))` # Output: Key not found

(vi) AttributeError

Occurs when trying to access an attribute that does not exist.

```
x = 10
```

```
x.append(5) # Integers do not have an append() method
```

Error Output:

AttributeError: 'int' object has no attribute 'append'

✓ **Fix:** Use correct data types.

(vii) ZeroDivisionError

Occurs when trying to divide a number by zero.

```
print(10 / 0)
```

Error Output:

ZeroDivisionError: division by zero

✓ **Fix:** Ensure the denominator is not zero before dividing.

(viii) FileNotFoundError

Occurs when trying to open a file that does not exist.

```
with open("nonexistent_file.txt", "r") as f:
```

```
    content = f.read()
```

Error Output:

FileNotFoundError: [Errno 2] No such file or directory: 'nonexistent_file.txt'

✓ **Fix:** Check if the file exists before opening.

(ix) ImportError / ModuleNotFoundError

Occurs when trying to import a module that does not exist.

```
import non_existent_module
```

Error Output:

ModuleNotFoundError: No module named 'non_existent_module'

✓ **Fix:** Install or check the module name.

try and except

The basic syntax is:

```
try:
```

```
    # Code that may produce an error
```

```
except:
```

```
    # Code to handle the error
```

Example

```
try:
```

```
    a = int(input("Enter a number: "))
```

```
    b = int(input("Enter another number: "))
```

```
    print(a / b)
```

```
except:
```

```
    print("Something went wrong")
```

If the user enters 10 and 0:

Something went wrong

Handling a Specific Error

It is better to specify the type of error.

ZeroDivisionError

try:

```
a = 10
```

```
b = 0
```

```
print(a / b)
```

except ZeroDivisionError:

```
print("Cannot divide by zero")
```

Output:

Cannot divide by zero

ValueError

ValueError occurs when a function receives an inappropriate value.

Example:

try:

```
number = int(input("Enter a number: "))
```

```
print(number)
```

except ValueError:

```
print("Please enter a valid number")
```

If the user enters:

abc

Output:

Please enter a valid number

NameError

A NameError occurs when we try to use a variable that has not been defined.

try:

```
print(name)
```

except NameError:

```
print("Variable is not defined")
```

Output:

Variable is not defined

TypeError

A TypeError occurs when an operation is performed on incompatible data types.

try:

```
a = 10
```

```
b = "20"
```

```
print(a + b)
```

except TypeError:

```
print("Cannot add integer and string")
```

Output:
Cannot add integer and string

IndexError

An IndexError occurs when we try to access an index that doesn't exist.

try:

```
numbers = [10, 20, 30]
```

```
print(numbers[5])
```

except IndexError:

```
print("Index does not exist")
```

Output:

Index does not exist

KeyError

A KeyError occurs when we try to access a dictionary key that doesn't exist.

try:

```
student = {  
    "name": "Abhinav",  
    "age": 24  
}
```

```
print(student["city"])
```

except KeyError:

```
print("Key does not exist")
```

Output:

Key does not exist

Multiple except Blocks

We can handle different errors separately.

try:

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
print(a / b)
```

except ValueError:

```
print("Please enter numbers only")
```

except ZeroDivisionError:

```
print("Cannot divide by zero")
```

else

The else block executes when **no error occurs**.

Syntax


```
try:
    # Code

except:
    # Error

else:
    # No error
```

Example

```
try:
    a = 10
    b = 2

    result = a / b

except ZeroDivisionError:
    print("Cannot divide by zero")

else:
    print("Result:", result)
Output:
Result: 5.0
```

finally

The finally block **always executes**, whether an error occurs or not.

Example

```
try:
    a = 10
    b = 0

    print(a / b)

except ZeroDivisionError:
    print("Cannot divide by zero")

finally:
    print("Program completed")
Output:
Cannot divide by zero
Program completed
```

try + except + else + finally

All four can be used together.

```
try:
    number = int(input("Enter a number: "))

except ValueError:
    print("Invalid input")
```

```
else:  
    print("You entered:", number)
```

```
finally:  
    print("Thank you")
```

Using as

We can store the error message in a variable using as.

```
try:  
    number = int("abc")
```

```
except ValueError as error:  
    print("Error:", error)
```

The error variable contains information about the exception.

raise

The raise statement is used to **manually generate an exception**.

Example

```
age = 15
```

```
if age < 18:  
    raise ValueError("Age must be 18 or above")
```

Output:

ValueError: Age must be 18 or above

raise with try-except

```
try:
```

```
    age = int(input("Enter your age: "))
```

```
    if age < 18:  
        raise ValueError("You must be 18 or above")
```

```
    print("Eligible")
```

```
except ValueError as error:  
    print("Error:", error)
```